

ARCHITECTURE N-TIERS ET DÉVELOPPEMENT WEB – LST GI –

Comprendre l'architecture n-tiers des applications web: le développement front-end avec HTML, CSS et JavaScript, et le back-end avec PHP. Mettre en œuvre la communication asynchrone via AJAX avec formats XML et JSON.



- 1) Architecture Web & environnement de travail
- 2) Développement Front-End (HTML & CSS)
- 3) Interactivité avec JavaScript
- 4) Communication Web (AJAX, JSON, XML)
- 5) Développement Back-End avec PHP

AU: 2025/2026

PR. AZROUMAH LI CHAÏMAE

1

AGENDA

- Introduction Générale
- Chapitre 1: Technologies Front-End (HTML, CSS, JavaScript)
- Chapitre 2: Technologies Back-End (PHP & Bases de données)
- **Chapitre 3: Architecture n-tiers & Intégration complète**
- TP : 25% de la note finale
- Examen final : 75% de la note finale

2

2

CHAPITRE 3: ARCHITECTURE N-TIERS & INTÉGRATION COMPLÈTE

SÉANCE 7

1) Architecture Logicielle

1. Introduction
2. La Base : Le Modèle Client-Serveur (Rappel)
3. L'Architecture N-Tiers (Le "multi-couches")
4. Restful API Design
5. Evolution : Les micro-services
- 2) Les design patterns
- 3) Intégration AJAX-PHP

3

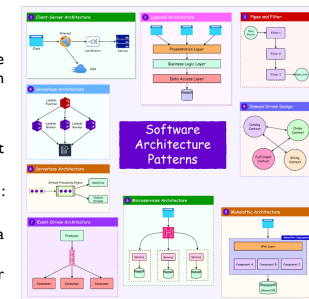
3

1) PARTIE 1: ARCHITECTURE LOGICIEL

1.1. INTRODUCTION

□ Introduction au Technologies Back-End

- **Architecture logicielle** : c'est le « plan de la ville ».
 - Elle définit la structure, l'organisation et les interactions entre les composants d'un système informatique, agissant comme un plan directeur.
- **Concepts clés et objectifs** :
 - ✓ **Structuration** : Découpage en modules et en couches, et définition des relations entre eux.
 - ✓ **Modèles (Patterns)** : représentations imagées (ex. : microservices, MVC) pour guider le développement.
 - ✓ **Performance et Sécurité** : Garantir la robustesse et la sécurité du système.
 - ✓ **Maintenance** : Rendre le code facile à comprendre, à modifier et à faire évoluer.



4

4

CHAPITRE 3: ARCHITECTURE N-TIERS & INTÉGRATION COMPLÈTE

SÉANCE 7

1) Architecture Logiciel

1. Introduction
 2. **La Base : Le Modèle Client-Serveur (Rappel)**
 3. L'Architecture N-Tiers (Le "multi-couches")
 4. Restful API Design
 5. Evolution: Les micro-services
- 2) Les design patterns
 - 3) Intégration AJAX-PHP

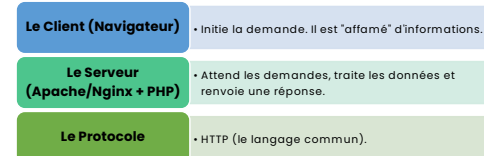
5

1) PARTIE 1: ARCHITECTURE LOGICIEL

1.2. LE MODÈLE CLIENT-SERVEUR

□ Rappel

- L'architecture client-serveur est un **modèle de communication informatique** où des postes clients demandent des **ressources** (données, applications) à un **serveur centralisé**, qui les traite et les fournit.

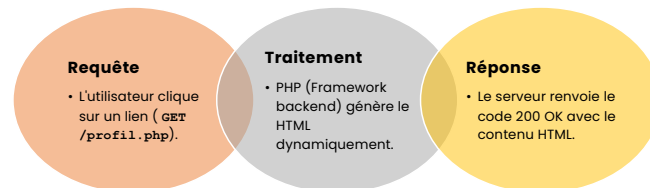


6

1) PARTIE 1: ARCHITECTURE LOGICIEL

1.2. LE MODÈLE CLIENT-SERVEUR

□ Le cycle de vie d'une requête :



▪ Exemple:

- ✓ Quand vous tapez google.com, votre navigateur (Client) envoie une demande au serveur de Google pour obtenir « la page d'accueil ».
- ✓ Le serveur répond en envoyant le pack HTML/JS.

7

CHAPITRE 3: ARCHITECTURE N-TIERS & INTÉGRATION COMPLÈTE

SÉANCE 7

1) Architecture Logiciel

1. Introduction
 2. La Base : Le Modèle Client-Serveur (Rappel)
 3. **L'Architecture N-Tiers (Le multi-couches)**
 4. Restful API Design
 5. Evolution: Les micro-services
- 2) Les design patterns
 - 3) Intégration AJAX-PHP

8

1) PARTIE 1: ARCHITECTURE LOGICIEL

1.3. L'ARCHITECTURE N-TIERS

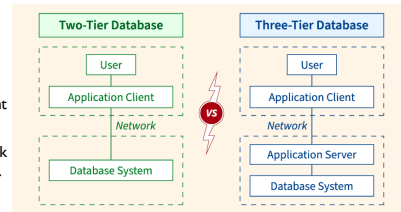
□ L'Architecture N-Tiers (Le "Multicouches")

- **Pourquoi "N" ?** On sépare l'application en plusieurs « couches » (Tiers) indépendantes pour que, si l'une d'elles tombe en panne ou change, les autres continuent de fonctionner.

▪ Exemple :

Imaginez un site d'e-commerce

- ✓ Le Client affiche le bouton « Acheter ».
- ✓ Le serveur PHP vérifie si le stock est suffisant et calcule les frais de port.
- ✓ La base de données réduit la quantité en stock dans la table des produits avant de les afficher.



9

9

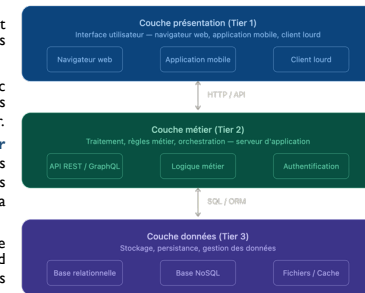
1) PARTIE 1: ARCHITECTURE LOGICIEL

1.3. L'ARCHITECTURE N-TIERS

□ Architecture 3 tier (Rappel)

- L'architecture 3-tiers est un modèle standard du développement web moderne qui divise une application en trois couches logiques distinctes :

- 1) **Couche 1 : Présentation (Front-end)** C'est l'interface avec laquelle l'utilisateur interagit directement. Elle affiche les données et collecte les entrées, sans contenir de logique métier.
- 2) **Couche 2 : Logique métier (Back-end / Serveur d'application)** C'est le cœur de l'application. Elle reçoit les requêtes du front-end, applique les règles métier, effectue les calculs, gère l'authentification et orchestre les échanges avec la base de données.
- 3) **Couche 3 : Données (Back-end de persistance).** Elle assure le stockage et la gestion des données. Elle répond uniquement aux requêtes du serveur d'application, jamais directement du front-end.



10

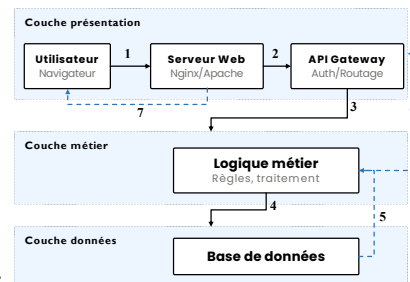
1) PARTIE 1: ARCHITECTURE LOGICIEL

1.3. L'ARCHITECTURE N-TIERS

□ Architecture 3 tier (Rappel)

Déclenchement côté client : l'utilisateur interagit avec l'interface. Le navigateur génère une requête HTTP ou HTTPS vers le serveur. Ex: GET /api/produits HTTP/1.1 | POST /api/commandes

- 1) **Réception par le serveur web :** le serveur reçoit la requête. Il peut servir les fichiers statiques directement ou transmettre les requêtes dynamiques à l'application.
- 2) **API Gateway : auth & routage :** L'API Gateway vérifie le token JWT ou la session, applique les règles de limitation (rate limiting), puis route la requête vers le bon micro-service ou le contrôleur.
- 3) **Traitement par la logique métier :** Le serveur d'application applique les règles métier : validation des données, calculs, orchestration des services. C'est ici que réside toute l'intelligence de l'application. SQL via une ORM ou des appels à des services externes (email, paiement...)



11

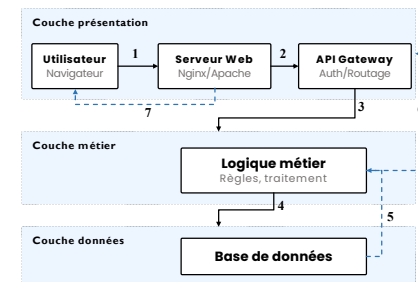
11

1) PARTIE 1: ARCHITECTURE LOGICIEL

1.3. L'ARCHITECTURE N-TIERS

□ Architecture 3 tier (Rappel)

- 4) **Requête en base de données :** La couche de données reçoit une requête SQL ou NoSQL, l'exécute, et retourne les résultats. La base ne communique jamais directement avec le client. Ex: `SELECT * FROM produits WHERE id = ? | db.collection.find()`
- 5,6,7) **Réponse remontante :** Les données remontent : base → logique métier (séréalisation JSON) → API Gateway (code HTTP) → serveur web → navigateur. Le client reçoit la réponse et met à jour l'interface. Ex: `HTTP 200 OK | Content-Type: application/json`



12

Exemple Explicatifs (Partie 1)

12

1) PARTIE 1: ARCHITECTURE LOGICIEL

1.3. L'ARCHITECTURE N-TIERS

□ Architecture 5 tier

- L'architecture à 5 tiers, une évolution de l'architecture à 3 tiers, est conçue pour les applications très complexes. Elle introduit deux couches intermédiaires supplémentaires par rapport à la **structure de base**.

- 1) **Couche Présentation (Client)** : Le navigateur gère uniquement l'affichage et les interactions de l'utilisateur.
- 2) **Couche Présentation Serveur (Web Tier)** : Un serveur web qui génère les vues ou gère les sessions
- 3) **Couche Logique Métier (Business Tier)** : C'est le cœur de l'application. Ici, on traite les règles complexes (calculs, validations de sécurité, algorithmes).
- 4) **Couche d'Accès aux Données (Data Access Tier)** : Elle fait le pont entre le code et la base de données. On y trouve souvent un ORM (Object-Relational Mapping).
- 5) **Couche de Données (Data Tier)** : La base de données physique ou les services de stockage externes.

```
graph TD
    subgraph Client_Side [Client Side]
        UI[User Interface Layer]
    end
    subgraph Server_Side [Server Side]
        BAL[Business Access Layer]
        BL[Business Logic Layer]
        DAL[Data Access Layer]
        DBL[Data base Layer]
    end
    UI --> BAL
    BAL --> BL
    BL --> DAL
    DAL --> DBL
```

13

13

1) PARTIE 1: ARCHITECTURE LOGICIEL

1.3. L'ARCHITECTURE N-TIERS

□ Architecture 5 tier

- **Exemple :**
- Pour bien faire comprendre la différence avec le 3-tiers :

1. **Tier 1 (Client)** : L'étudiant remplit un formulaire de virement depuis son téléphone (React/Vue.js).
2. **Tier 2 (Web)** : Un serveur de passerelle (API Gateway) reçoit la demande et vérifie que l'étudiant est bien connecté.
3. **Tier 3 (Business)** : Un service spécifique vérifie si le solde est suffisant et calcule les frais bancaires en fonction du pays.
4. **Tier 4 (Data Access)** : Une couche spécialisée traduit l'ordre de virement en une requête SQL complexe ou communique avec une API bancaire externe (comme Swift).
5. **Tier 5 (Data)** : La base de données Oracle ou MySQL enregistre la transaction finale.

Exemple Explicatifs (Partie 1)

14

14

CHAPITRE 3: ARCHITECTURE N-TIERS & INTÉGRATION COMPLÈTE

SÉANCE 7

- 1) **Architecture Logiciel**
 1. Introduction
 2. La Base : Le Modèle Client-Serveur (Rappel)
 3. L'Architecture N-Tiers (le "multi-couches")
 4. **Restful API Design**
 5. Evolution : Les micro-services
- 2) Les design patterns
- 3) Intégration AJAX-PHP

15

15

1) PARTIE 1: ARCHITECTURE LOGICIEL

1.4. RESTFUL API DESIGN

- Le design RESTful de l'API est une architecture permettant la **communication standardisée entre applications, sans état (stateless)**, en utilisant les **verbes HTTP**.
 - Les ressources sont identifiées par une URL (par exemple : /utilisateurs/42).
- Les méthodes principales sont : GET (lire une ressource), POST (créer une ressource), PUT (modifier une ressource) et DELETE (supprimer une ressource).
- L'échange est généralement effectué au format JSON, un format facile à lire en JavaScript.
- **Exemple :**

Vous développez une application mobile en JS qui doit récupérer des articles stockés sur un serveur PHP.

- ✓ Le JS appelle : `fetch('api/articles.php')`.
- ✓ Le PHP ne renvoie pas du HTML, mais du texte brut au format JSON : `{"id": 1, "titre": "Mon premier article"}`.
- ✓ L'application mobile traite ce JSON pour l'afficher.

16

16

CHAPITRE 3: ARCHITECTURE N-TIERS & INTÉGRATION COMPLÈTE

SÉANCE 7

1) Architecture Logiciel

1. Introduction
 2. La Base : Le Modèle Client-Serveur (Rappel)
 3. L'Architecture N-Tiers (Le "multi-couches")
 4. Restful API Design
 5. Evolution : Les micro-services
- 2) Les design patterns
 - 3) Intégration AJAX-PHP

17

17

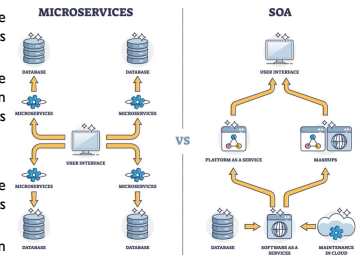
1) PARTIE 1: ARCHITECTURE LOGICIEL

1.5. LES MICRO-SERVICES

- **Quand le N-Tier ne suffit plus**
- Où en parler ? C'est l'étape après le N-Tier. Au lieu d'avoir une seule grosse application monolithique, on la découpe en plusieurs petites applications indépendantes.
- C'est un style d'architecture logicielle qui décompose une application en un ensemble de petits services autonomes, chacun responsable d'une fonction métier précise, déployables et scalables de manière indépendante.

□ Un micro-service vs un monolithe

- La différence fondamentale est visible d'un coup d'œil : dans le monolithe, toutes les fonctionnalités partagent le même processus et la même base de données.
- Dans l'architecture microservices, chaque service est un programme indépendant avec sa propre base de données.



18

18

1) PARTIE 1: ARCHITECTURE LOGICIEL

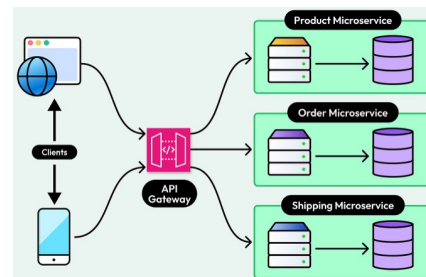
1.5. LES MICRO-SERVICES

□ Quand le N-Tier ne suffit plus

■ Exemple :

- Un micro-service pour la gestion des produits, un autre pour les demandes, un autre pour la livraison.

→ Avantage : Si le service « Livraison » tombe en panne, les clients peuvent toujours consulter le « Catalogue ».



Exemple Explicatifs (Partie 1)

19

19

CHAPITRE 3: ARCHITECTURE N-TIERS & INTÉGRATION COMPLÈTE

SÉANCE 7

1) Architecture Logiciel

2) Les design pattern

1. Introduction
2. MVC: Modèle Vue Contrôleur
3. Autre design pattern
- 3) Intégration AJAX-PHP

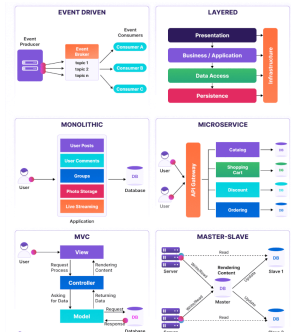
20

20

2) PARTIE 2: LES DESIGN PATTERN

2.1. INTRODUCTION

- Les design patterns (patrons de conception) : C'est le "plan d'un appartement".
- C'est une solution standard à un problème de code récurrent dans un bloc.
- Exemple :** Comment organiser mes fichiers HTML/CSS/JS/PHP pour qu'ils ne soient pas illisibles ?



21

CHAPITRE 3: ARCHITECTURE N-TIERS & INTÉGRATION COMPLÈTE

SÉANCE 7

- 1) Architecture Logicielle
- 2) Les design pattern
 1. Introduction
 2. MVC: Modèle Vue Contrôleur
 3. Autre design pattern
- 3) Intégration AJAX-PHP

22

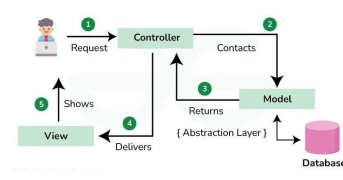
2) PARTIE 2: LES DESIGN PATTERN

2.2. MVC: MODÈLE VUE CONTRÔLEUR

- Le modèle MVC (Modèle-Vue-Contrôleur) est une architecture logicielle qui structure une application en trois couches distinctes pour séparer la logique métier, l'interface utilisateur et la gestion des requêtes: le modèle gère les données, la vue s'occupe de l'affichage, et le contrôleur fait l'interface entre les deux.
- Organiser le code interne du serveur (Ex : PHP) pour qu'il ne devienne pas un « plat de spaghettis ».

Les 3 composants de l'Architecture MVC

- Modèle (Model) :** Gère les données, la logique métier et les règles de l'application.
 - Il interagit avec la base de données (via des requêtes SQL) pour y stocker ou y récupérer des informations. Il ne connaît pas la vue.
- Vue (View) :** Gère l'affichage et l'interface utilisateur.
 - Elle présente à l'utilisateur les données fournies par le contrôleur (généralement sous forme d'HTML).
- Contrôleur (Controller) :** C'est l'interface entre le modèle et la vue.
 - Il reçoit les actions de l'utilisateur (clics, soumission de formulaires), demande au modèle de mettre à jour les données, puis choisit la vue à afficher.

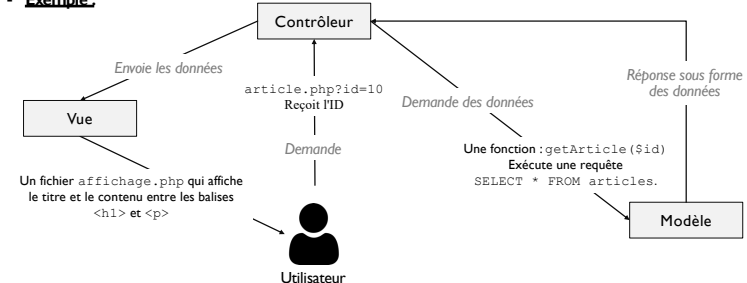


23

2) PARTIE 2: LES DESIGN PATTERN

2.2. MVC: MODÈLE VUE CONTRÔLEUR

Exemple :



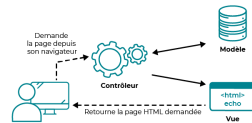
24

2) PARTIE 2: LES DESIGN PATTERN

2.2. MVC: MODÈLE VUE CONTRÔLEUR

□ Fonctionnement du flux MVC

- 1) **Utilisateur** : interagit avec la vue (ex. : clique sur un lien).
- 2) **Contrôleur** : reçoit la requête, traite les données et demande une mise à jour du modèle.
- 3) **Modèle** : Modifie ou récupère les données dans la base de données.
- 4) **Contrôleur** : récupère les données du modèle et les transmet à la vue.
- 5) **Vue** : Met à jour l'affichage final pour l'utilisateur.



□ Avantages du MVC

- **Séparation des préoccupations** : Le code est mieux organisé, en séparant la logique métier de l'affichage.
- **Maintenance facilitée** : Il est facile de modifier l'interface (vue) sans toucher à la logique des données (modèle).
- **Réutilisabilité** : Les modèles peuvent être réutilisés pour différentes vues (ex. : une API et une page web).
- **Travail d'équipe** : Les développeurs backend (modèle) et frontend (vue) peuvent travailler en parallèle.

Exemple Explicatifs (Partie 2)

25

CHAPITRE 3: ARCHITECTURE N-TIERS & INTÉGRATION COMPLÈTE

SÉANCE 7

- 1) Architecture Logicielle
- 2) Les design pattern
 1. Introduction
 2. MVC: Modèle Vue Contrôleur
 3. Autre design pattern
- 3) Intégration AJAX-PHP

26

2) PARTIE 2: LES DESIGN PATTERN

2.3. LES AUTRES DESIGN PATTERN

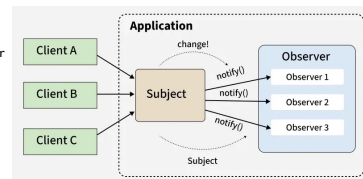
□ Le Pattern "Observer" (L'Observateur)

- Dans ce pattern, un objet (le « Sujet ») prévient automatiquement d'autres objets (les "Observateurs") dès qu'il change d'état.
- Elle est idéale pour les systèmes d'événements ou de notifications.
- L'Avantage de ce pattern est le découplage du code.
- Le système d'inscription n'a pas besoin de savoir comment envoyer un mail, il crie juste "Un utilisateur s'est inscrit !" et les autres modules réagissent.

▪ Exemple :

Sur un site e-commerce, quand un client clique sur « Valider commande » :

- Le module Stock réduit les quantités de produits.
- Le module Email envoie la confirmation.
- Le module Logistique imprime l'étiquette.
- Tous « observent » l'événement `CommandeValidée`.

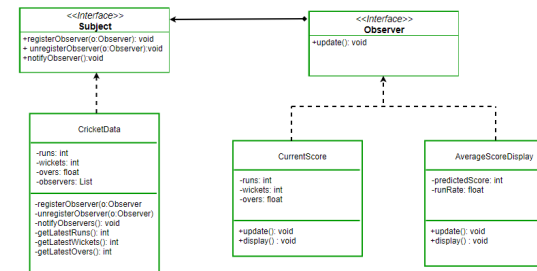


27

2) PARTIE 2: LES DESIGN PATTERN

2.3. LES AUTRES DESIGN PATTERN

▪ Exemple :



Exemple Explicatifs (Partie 2)

28

27

28

2) PARTIE 2: LES DESIGN PATTERN

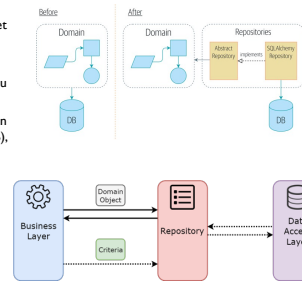
2.3. LES AUTRES DESIGN PATTERN

□ Le Pattern "Repository" (Le Bibliothécaire)

- Dans ce pattern, l'idée clé est de faire l'intermédiaire entre la logique métier et la source de données (SQL).
- C'est le cœur du Tier 4 dans une architecture à 5 tiers.
- L'utilité de ce pattern est qu'on ne veut pas voir de `SELECT * FROM...` au milieu de notre logique de calcul.
- L'avantage majeur de ce pattern est que si, pour une raison ou une autre, on veut changer le serveur de la base de données (ex. : de MySQL à MongoDB), on ne change que le repository, et pas notre logique métier.

■ Exemple :

- Au lieu d'écrire du SQL dans votre contrôleur, vous appelez `$userRepository->findAllActive()`.
- Le contrôleur demande les données, le Repository sait comment aller les chercher.



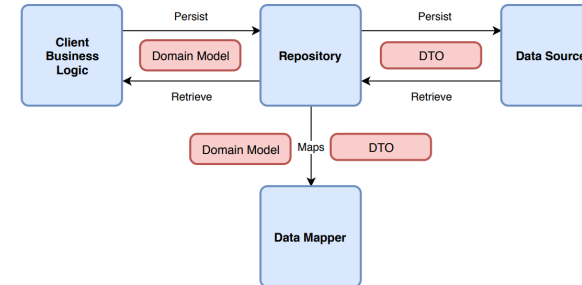
29

29

2) PARTIE 2: LES DESIGN PATTERN

2.3. LES AUTRES DESIGN PATTERN

■ Exemple :



Exemple Explicatifs (Partie 2)

30

30

2) PARTIE 2: LES DESIGN PATTERN

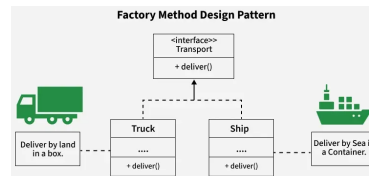
2.3. LES AUTRES DESIGN PATTERN

□ Le Pattern « Factory » (Le Bibliothécaire)

- L'idée clé du pattern Factory consiste à créer des objets sans exposer la logique de création au client.
- On délègue la fabrication à une « usine ».
- L'utilité principale de ce pattern se manifeste lorsque vous avez plusieurs types d'objets similaires mais présentant des comportements différents.

■ Exemple :

- Un système de notifications:
 - Selon le choix de l'utilisateur, l'usine `NotificationFactory` va fabriquer soit un objet `EmailNotification`, soit un objet `SMSNotification`.
 - Le reste de votre code utilise simplement une méthode `.send()` sans se soucier du type de média.



31

31

CHAPITRE 3: ARCHITECTURE N-TIERS & INTÉGRATION COMPLÈTE

SÉANCE 7

- 1) Architecture Logiciel
- 2) Les design pattern
- 3) Intégration AJAX-PHP
 1. Communication bidirectionnelle
 2. Requêtes asynchrones
 3. Réponses JSON
 4. Mise à jour dynamique
 5. Gestion erreurs

32

32

3) PARTIE 3: INTÉGRATION AJAX-PHP

3.1. LA COMMUNICATION BIDIRECTIONNELLE

□ Rappel à AJAX (Rappel)

- AJAX (Asynchronous JavaScript and XML) est un paradigme de communication client-serveur non bloquant, permettant :
 - ✓ d'envoyer des requêtes HTTP en arrière-plan
 - ✓ de recevoir des données
 - ✓ de modifier le DOM dynamiquement sans recharger la page
- AJAX n'est pas une technologie unique mais un ensemble de technologies :
 - ▶ JavaScript, HTTP, DOM, JSON/XML, serveur (PHP)

Workflow AJAX détaillé

```

    graph LR
    A[User Action] --> B[JS Event]
    B --> C[AJAX Request]
    C --> D[PHP]
    D --> E[DB]
    E --> F[JSON Response]
    F --> G[DOM Update]
    
```

33

33

3) PARTIE 3: INTÉGRATION AJAX-PHP

3.1. LA COMMUNICATION BIDIRECTIONNELLE

□ Communication Bidirectionnelle

- Le concept technique:** La communication bidirectionnelle signifie :
 - Que le client envoie une requête, et le serveur répond avec des données exploitables
- La structure d'échange:**
 - **Requête:**
 - ✓ URL: POST
 - ✓ Method: /api/users
 - ✓ Content-Type: application/json
 - **Réponse:**
 - ✓ { "status": "success" }
- Exemple:**
 - **Requête : Frontend JS**

```

                    fetch("/api/login", {
                      method: "POST",
                      body: JSON.stringify({
                        email, password
                      })
                    })
                
```
 - **Réponse : Backend PHP**

```

                    if($valid){
                      echo json_encode(["login">true]);
                    }
                
```

34

34

3) PARTIE 3: INTÉGRATION AJAX-PHP

3.1. LA COMMUNICATION BIDIRECTIONNELLE

□ Communication Bidirectionnelle

- La communication bidirectionnelle désigne un échange de données où chaque entité (client et serveur) peut à la fois émettre et recevoir des informations.
 - ▶ Dans le contexte d'AJAX, cela signifie que le navigateur peut envoyer des données au serveur sans attendre un rechargement de page, et le serveur peut répondre avec des données que le navigateur traite immédiatement.

Dans le modèle web traditionnel (synchrone) :

- 1) Le client envoie une requête.
- 2) Le serveur répond avec une page complète.
- 3) Le client ne peut plus interagir tant que la réponse n'est pas reçue.

Avec AJAX :

- 1) Le client envoie une requête en arrière-plan.
- 2) Le serveur répond avec des données (pas une page).
- 3) Le client peut continuer à interagir avec l'interface pendant l'attente.
- 4) Le client peut renvoyer de nouvelles requêtes en fonction des réponses reçues.
- 5) **Cette bidirectionnalité est asynchrone :** les échanges se font indépendamment du flux principal d'interaction.

35

35

3) PARTIE 3: INTÉGRATION AJAX-PHP

3.1. LA COMMUNICATION BIDIRECTIONNELLE

▪ Syntaxe (côté client) :

```

// Envoi d'une requête avec envoi de données (client → serveur)
fetch('api.php', {
  method: 'POST',
  body: JSON.stringify({ message: "Hello serveur" })
})
.then(response => response.json())
.then(data => {
  // Réception et traitement de la réponse (serveur → client)
  console.log("Le serveur a répondu :", data);
});
    
```

36

36

CHAPITRE 3: ARCHITECTURE N-TIERS & INTÉGRATION COMPLÈTE

SÉANCE 7

- 1) Architecture Logiciel
- 2) Les design pattern
- 3) **Intégration AJAX-PHP**
 1. Communication bidirectionnelle
 2. **Requêtes asynchrones**
 3. Réponses JSON
 4. Mise à jour dynamique
 5. Gestion erreurs

37

3) PARTIE 3: INTÉGRATION AJAX-PHP

3.2. LES REQUÊTES ASYNCHRONES

- Une requête asynchrone est une requête HTTP exécutée sans bloquer le thread principal d'exécution du navigateur.
- Le code JavaScript continue de s'exécuter pendant que la requête est en cours, et un mécanisme de rappel (callback), de Promise ou d'async/await permet de traiter la réponse lorsqu'elle est disponible.

- Synchrone** (à éviter en JavaScript) :

```
// Bloque tout jusqu'à réception de la réponse
const reponse = attendreResultat(); // imaginaire
console.log(reponse);
console.log("Ceci attendra");
```

- Asynchrone** (recommandé) :

```
console.log("Début");
fetch('api.php')
  .then(response => response.json())
  .then(data => console.log(data));
console.log("Fin"); // S'affiche avant la réponse
```

La navigation n'est jamais interrompue. L'utilisateur peut :

- Saisir du texte dans d'autres champs
- Cliquer sur d'autres éléments
- Voir un indicateur de chargement

38

3) PARTIE 3: INTÉGRATION AJAX-PHP

3.2. LES REQUÊTES ASYNCHRONES

□ Syntaxe des requêtes asynchrones:

- Méthode 1:** Promesses avec .then()

```
fetch('traitement.php', { method: 'POST', body: donnees })
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

- Méthode 2:** Async/Await (recommandé pour la lisibilité)

```
async function envoyerRequete() {
  try {
    const response = await fetch('traitement.php', {
      method: 'POST',
      body: donnees });
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
```

- Méthode 3:** Avec gestion d'état de chargement

```
async function rechercher() {
  // Affiche un indicateur
  afficherLoader(true);
  try {
    const response = await
      fetch('recherche.php');
    const data = await response.json();
    afficherResultats(data);
  } finally {
    // Cache l'indicateur
    afficherLoader(false);
  }
}
```

39

39

CHAPITRE 3: ARCHITECTURE N-TIERS & INTÉGRATION COMPLÈTE

SÉANCE 7

- 1) Architecture Logiciel
- 2) Les design pattern
- 3) **Intégration AJAX-PHP**
 1. Communication bidirectionnelle
 2. Requêtes asynchrones
 3. **Réponses JSON**
 4. Mise à jour dynamique
 5. Gestion erreurs

40

40

3) PARTIE 3: INTÉGRATION AJAX-PHP

3.3. LES RÉPONSES JSON

Format JSON (Rappel) :

- JSON (JavaScript Object Notation) est un format léger d'échange de données, structuré sous forme de paires clé-valeur et de listes.
- Il est nativement interprétable par JavaScript, ce qui en fait le format privilégié pour AJAX.
- Les avantages de JSON incluent la lisibilité, le fait qu'elle est légère comparée à XML, il a un type simple (chaînes, nombres, booléens, tableaux, objets) et finalement, elle est interopérable entre tous les langages (PHP, Java, Python, etc.).

Format JSON typique :

```
{
  "status": "success",
  "data": {
    "id": 42,
    "nom": "Architecture 5 tiers",
    "tags": ["web", "backend"]
  },
  "message": "Opération réussie"
}
```

41

41

3) PARTIE 3: INTÉGRATION AJAX-PHP

3.3. LES RÉPONSES JSON

Syntaxe des réponses JSON

Côté PHP (encodage)

```
<?php
$donnees = [
  'status' => 'success',
  'utilisateurs' => [
    ['id' => 1, 'nom' => 'Alice'],
    ['id' => 2, 'nom' => 'Bob']
  ];

header('Content-Type:
application/json');
echo json_encode($donnees);
exit;
?>
```

Côté JavaScript (décodage)

```
// Avec Fetch API (décodage automatique)
const response = await fetch('api.php');
// Parse automatiquement le JSON
const data = await response.json();

// Avec XMLHttpRequest (ancienne méthode)
const xhr = new XMLHttpRequest();
xhr.onload = function() {
  const data = JSON.parse(xhr.responseText);
};
```

42

42

CHAPITRE 3: ARCHITECTURE N-TIERS & INTÉGRATION COMPLÈTE

SÉANCE 7

- 1) Architecture Logiciel
- 2) Les design pattern
- 3) Intégration AJAX-PHP
 1. Communication bidirectionnelle
 2. Requêtes asynchrones
 3. Réponses JSON
4. Mise à jour dynamique
 5. Gestion erreurs

43

43

3) PARTIE 3: INTÉGRATION AJAX-PHP

3.4. MISE À JOUR DYNAMIQUE

- La mise à jour dynamique consiste à modifier le contenu d'une page web existante sans rechargement, en manipulant directement le DOM (Document Object Model) à partir des données reçues via AJAX.

Principes fondamentaux :

Sélection des éléments à modifier
(getElementById, querySelector)

Modification du contenu
(innerHTML, textContent, création d'éléments)

Animation optionnelle pour une meilleure expérience utilisateur

Approches :

Remplissage : injecter du HTML généré à partir des données

Ajout/Suppression : ajouter ou retirer des éléments de la liste

Attributs : modifier des classes, styles, ou attributs (disabled, hidden)

44

44

3) PARTIE 3: INTÉGRATION AJAX-PHP

3.4. MISE À JOUR DYNAMIQUE

□ Syntaxe de base pour la manipulation de DOM

```
// Modifier le texte
document.getElementById('status').textContent = 'Chargement terminé';

// Modifier le HTML
document.getElementById('contenu').innerHTML = '<strong>Nouveau contenu</strong>';

// Créer et ajouter un élément
const nouvelElement = document.createElement('div');
nouvelElement.className = 'item';
nouvelElement.textContent = 'Nouvel élément';
document.getElementById('liste').appendChild(nouvelElement);

// Supprimer tous les enfants
const container = document.getElementById('liste');
while (container.firstChild) {
    container.removeChild(container.firstChild);
}
```

45

45

3) PARTIE 3: INTÉGRATION AJAX-PHP

3.4. MISE À JOUR DYNAMIQUE

□ Syntaxe de base pour la génération dynamique à partir des données JSON:

```
function afficherProduits(produits) {
    const container = document.getElementById('catalogue');
    container.innerHTML = ''; // Nettoyage
    produits.forEach(produit => {
        const card = document.createElement('div');
        card.className = 'produit-card';
        card.innerHTML = `
            <h3>${escapeHtml(produit.nom)}</h3>
            <p>Prix: ${produit.prix} €</p>
            <button onclick="ajouterPanier(${produit.id})">Ajouter</button>
        `;
        container.appendChild(card);
    });
}
```

46

46

CHAPITRE 3: ARCHITECTURE N-TIERS & INTÉGRATION COMPLÈTE

SÉANCE 7

- 1) Architecture Logiciel
- 2) Les design pattern
- 3) Intégration AJAX-PHP
 1. Communication bidirectionnelle
 2. Requêtes asynchrones
 3. Réponses JSON
 4. Mise à jour dynamique
 5. Gestion erreurs

47

47

3) PARTIE 3: INTÉGRATION AJAX-PHP

3.5. GESTION DES ERREURS

- La **gestion des erreurs** (ou error handling) est l'ensemble des mécanismes mis en place pour détecter, traiter et communiquer les problèmes survenant lors d'une communication AJAX, qu'ils soient d'origine réseau, serveur ou applicative.

□ Types d'erreurs et leurs origines

Type	Origine	Exemples
Réseau	Navigateur / connexion	Hôte inaccessible, CORS, timeout, DNS
HTTP	Serveur (protocole)	404 Not Found, 403 Forbidden, 500 Internal Server Error
Applicative	Serveur (logique métier)	Données invalides, authentification requise, ressource introuvable

□ Stratégies de gestion

- 1) **Détection** : identifier l'erreur via try/catch, response.ok, ou les codes HTTP.
- 2) **Communication** : informer l'utilisateur avec des messages clairs et non techniques.
- 3) **Journalisation** : enregistrer les erreurs côté serveur pour débogage.
- 4) **Récupération** : proposer une action (réessayer, se reconnecter, etc.).

48

48

3) PARTIE 3: INTÉGRATION AJAX-PHP

3.5. GESTION DES ERREURS

□ Syntaxe de gestion des erreurs : coté client (1/3)

```
async function requeteAvecGestionErreurs(url, options) {
  try {
    const response = await fetch(url, options);
    // Vérification du statut HTTP
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}: ${response.statusText}`);
    }
    const data = await response.json();
    // Vérification du statut applicatif
    if (data.status === 'error') {
      throw new Error(data.message);
    }
    return data;
  } catch (error) {
    ...
  }
}
```

49

49

3) PARTIE 3: INTÉGRATION AJAX-PHP

3.5. GESTION DES ERREURS

□ Syntaxe de gestion des erreurs : coté client (2/3)

```
async function requeteAvecGestionErreurs(url, options) {
  try { ... }
  catch (error) {
    // Gestion différenciée selon le type d'erreur
    if (error.name === 'TypeError' && error.message.includes('fetch')) {
      afficherErreurUtilisateur('Problème de connexion. Vérifiez votre réseau.');
```

50

50

3) PARTIE 3: INTÉGRATION AJAX-PHP

3.5. GESTION DES ERREURS

□ Syntaxe de gestion des erreurs : coté client (3/3)

```
function afficherErreurUtilisateur(message) {
  const zoneErreur = document.getElementById('erreur');
  zoneErreur.textContent = message;
  zoneErreur.style.display = 'block';
  setTimeout(() => {
    zoneErreur.style.display = 'none';
  }, 5000);
}
```

51

51

3) PARTIE 3: INTÉGRATION AJAX-PHP

3.5. GESTION DES ERREURS

□ Syntaxe de gestion des erreurs : coté serveur (1/2)

```
<?php
// Structure de réponse standardisée
function reponseSucces($donnees, $message = '') {
  return [
    'status' => 'success',
    'data' => $donnees,
    'message' => $message
  ];

function reponseErreur($message, $code = 400) {
  http_response_code($code);
  return [
    'status' => 'error',
    'message' => $message,
    'data' => null
  ];
} ...
```

52

52

3) PARTIE 3: INTÉGRATION AJAX-PHP

3.5. GESTION DES ERREURS

□ Syntaxe de gestion des erreurs : coté serveur (2/2)

```
// Exemple d'utilisation
try {
    $input = json_decode(file_get_contents('php://input'), true);
    if (!isset($input['email'])) {
        echo json_encode(reponseErreur('Email requis', 400));
        exit;
    }
    if (!filter_var($input['email'], FILTER_VALIDATE_EMAIL)) {
        echo json_encode(reponseErreur('Email invalide', 400));
        exit;
    }
    // Traitement...
    echo json_encode(reponseSucces(['id' => 123], 'Utilisateur créé'));
} catch (PDOException $e) {
    error_log($e->getMessage()); // Journalisation
    echo json_encode(reponseErreur('Erreur base de données', 500));
}
?>
```

Exemple Explicatifs (Partie 3)

53

53

ARCHITECTURE N-TIERS ET DÉVELOPPEMENT WEB – LST GI –



Comprendre l'architecture n-tiers des applications web: le développement front-end avec HTML, CSS et JavaScript, et le back-end avec PHP. Mettre en œuvre la communication asynchrone via AJAX avec formats XML et JSON.

Fin de cours

AU: 2025/2026

PR. AZROUMAH LI CHAIMAE

54